Software Dependability

Dependability Requirements, Faults, and Treatments

Software Dependability - Definition

Dependability: *“[…] the trustworthiness of a computing system which allows reliance to be justifiably placed on the service it delivers […]”.*

IFIP 10.4 Working Group on Dependable Computing and Fault Tolerance

The dependability of a system is its ability to prevent service failures that are more frequent and more severe than acceptable. It reflects the extent of the user’s confidence when operating and interacting with it.

A system can be considered dependable if it operates without interruption, delivers the services that the stakeholders expect, does not adversely affect the system’s environment, and does not damage its data or the system itself.

Failures

A service is correctly delivered when it implements its functions.

A service failure, i.e., a failure, is an event that occurs when the delivered service deviates from the correct service.

The consequences of failures are serious!

We need precise definitions of how dependable our systems must be. We cannot make them perfect.
If we overlook something, the system might fail, and no one will care how well the rest of it performed.









Why do Failures Occur?

There are two conditions:
1. A failure occurs when a system enters an erroneous state.
2. Part of that erroneous state is reflected in the system's external state.

An erroneous state, or error, is a system state that could lead to a service failure.

Since preventing the second condition is difficult, our goal is to avoid errors by:
1. Identifying all possible erroneous states the system could reach,
2. Ensuring these states are either unreachable or, if reachable, their adverse effects are known and minimized through every appropriate measure (ALARP).

"A fault is the adjudged or hypothesized cause of an error." It can concern hardware or software.

Failures and Faults

“A fault is the adjudged or hypothesized cause of an error.” It can concern hardware or software.

Faults do occur
and those faults lead to errors
and those errors lead to service failures.

We must understand faults completely and thoroughly because we must address them fully.

Not all faults are the same! They can relate to the specifications, the design, and the implementation of a system.
Also, their effects can vary depending on different perspectives. In this case, we talk about Byzantine faults.

A fault can be
- **Active**, if it causes an error.
- **Dormant**, unless it causes an error.
- **Permanent**, if it is present and remains present.
- **Transient**, if it is present but does not remain present.

Some faults could even cause the same software to not always fail on the same input. In this case, we talk about **Heisenbugs**.

From Fault to Hazards

A hazardous state, often called a hazard, is a system condition that could result in an unplanned loss of life, property damage, energy release, or the release of dangerous materials.

An accident is a loss event.

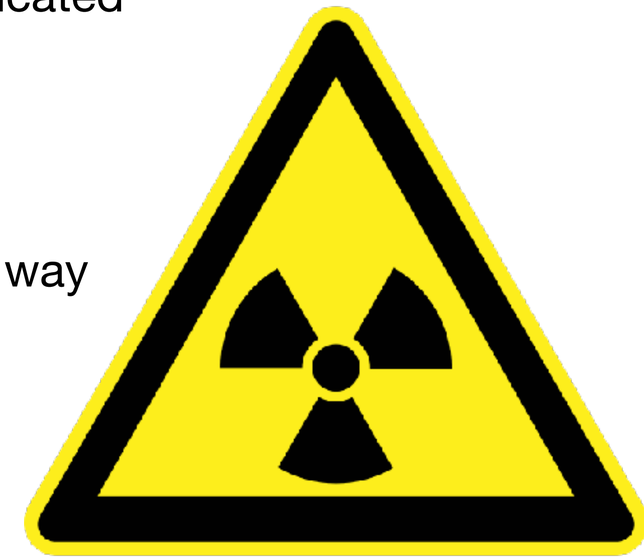
A system can enter a hazardous state for two reasons:

- The system design includes hazardous states.
- A component failure causes the system to enter a hazardous state.

When designing a system, it is essential to identify potential hazards to understand how to eliminate them effectively. Hazard identification and analysis are complex and sophisticated processes performed by engineers who are experts in the application domain.

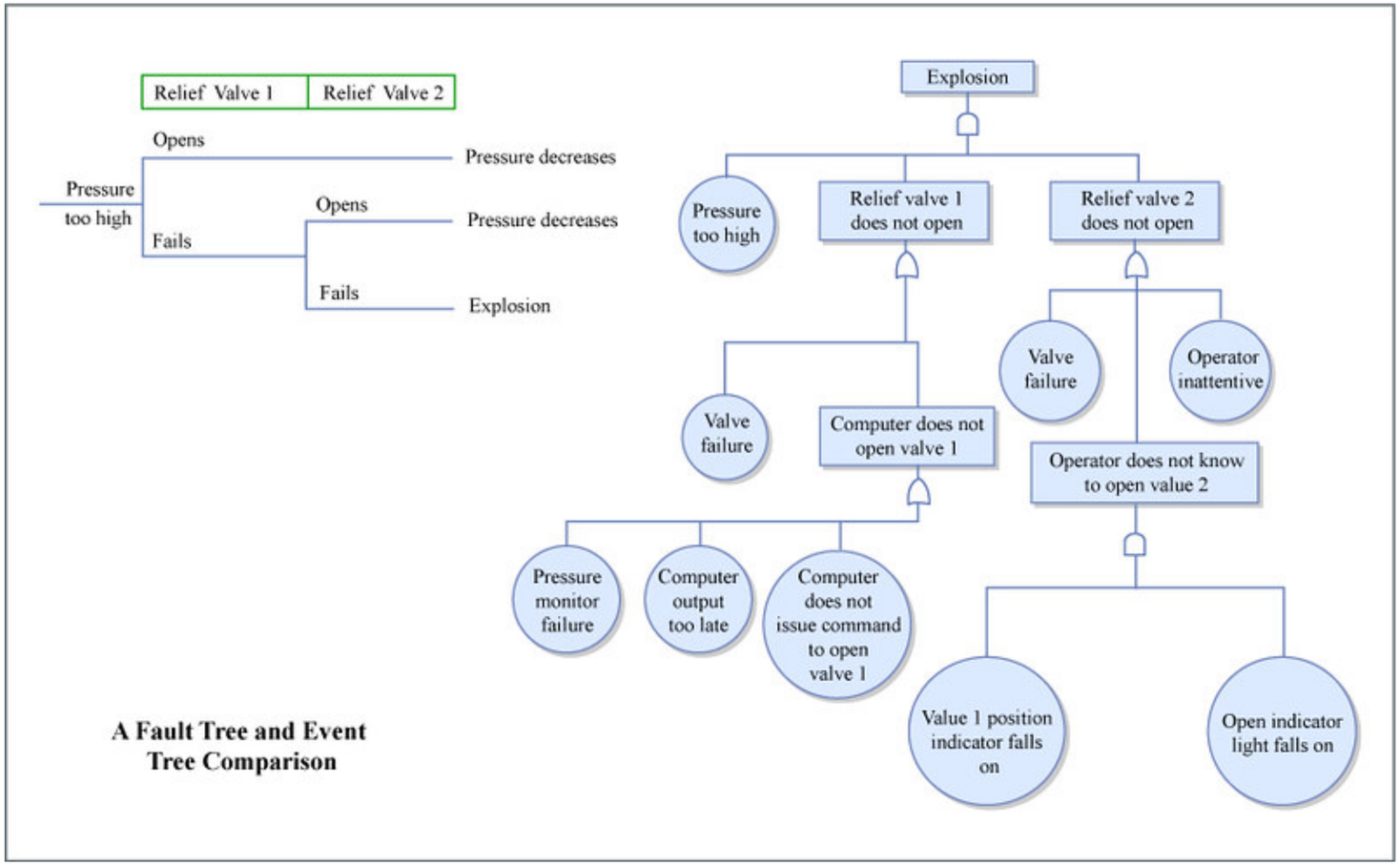
Hazards are errors.

The combination of fault identification and hazard analysis provides a comprehensive way to identify the faults to which a system is subject.



An Example of Hazard Analysis

A Fault Tree is a directed acyclic graph made up of events and gates, where events are occurrences like subsystem failures and gates illustrate how failures spread through the system by combining input events. The top event in the tree represents the failure of the system being analyzed.



A Fault Tree and Event Tree Comparison



The Fundamental Principle of Dependability

“If all faults to which a system is subject are identified and their effects are properly mitigated, the system will meet its dependability requirements.”

There are different ways to mitigate faults; they can be

- **Avoided**: If we build systems without faults, we do not need to do anything else.
- **Eliminated**: Do you remember software testing?
- **Tolerated**: Managed in a way that they do not cause failures.
- **Forecast**: Suppose when their effects will occur and be prepared for them.

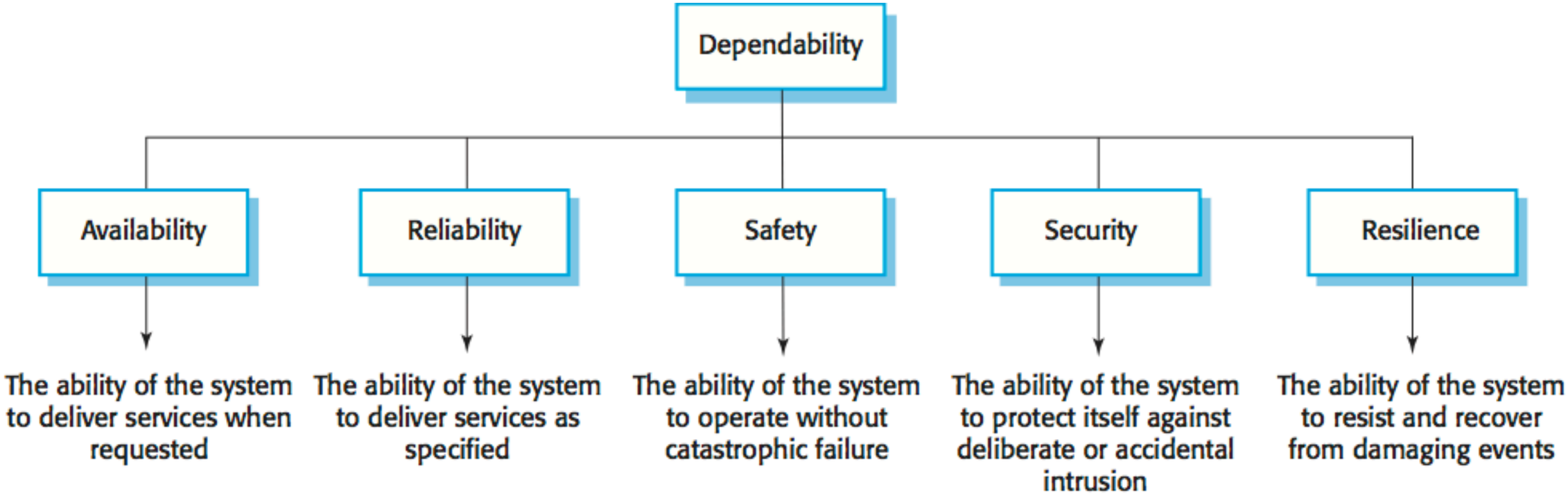
Dependability Engineering - Goals

- Software Dependability **ensures that designs meet product dependability requirements**.
- Software Dependability **verifies that a product will function reliably over its mission lifetime**.
- Software Dependability **identifies design discrepancies and solves them**.
- Software Dependability **evaluates potential failures and their effects on the mission, guiding corrective actions**.
- Software Dependability **recommends design configurations for redundancy**.
- Software Dependability **establishes cost-effective test plans based on the reliability goals**.
- Software Dependability **assesses failure probability at mission lifetime**.
- Software Dependability **predicts reliability and availability**.

Costs connected to dependability



A Closer Look to the Dependability Attributes and Their Meaning



Software Availability

To be usable, a software system must be available, meaning it must be accessible to users most of the time.

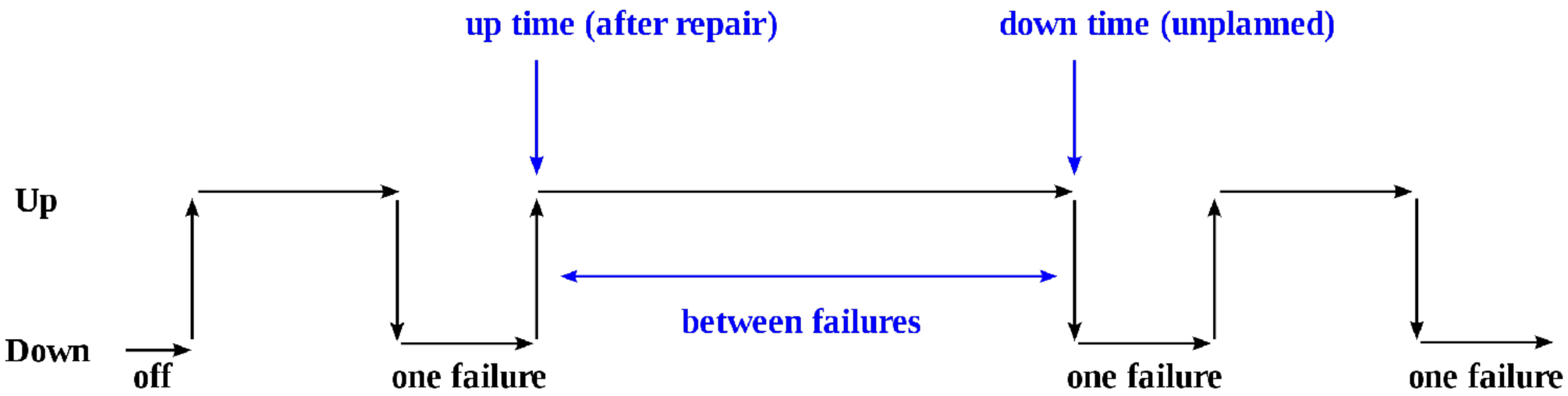
This aspect is closely related to fault tolerance and resilience: software is considered available if it does not fail.

Availability can be further enhanced through data redundancy techniques and fault recovery strategies.

System availability can be measured by counting the frequency of successful system operations.
A well-known metric for this is $(1 - (downtime/totaltime))$.

In our course, we will indirectly examine availability by focusing on methods to manage issues that could threaten it.

Software Availability



Repairable and Non-repairable Items

Non-repairable items refer to products that cannot be repaired after they break down or are not cost-effective to fix. Examples of non-repairable items in everyday life include light bulbs and CPUs. The reliability of non-repairable items is the probability that an item will survive a specific time period without failing. In this context, we focus on the likelihood of the first failure, since the item cannot be repaired afterward.

Repairable items are products that can be economically fixed after breaking down, or where the repair costs are significantly less than the original purchase price. Examples include cars, washing machines, and dishwashers. The reliability of repairable items refers to the probability that a failure will not occur within a given period. These items can be repaired multiple times during their useful life.

NB: Availability applies to
repairable and non-repairable systems.

Software Availability

Availability refers to the probability that a system is functioning properly when it is **requested for use**.

For example, a lamp with 99.90% availability means that, on average, there will be once in every one thousand occasions when someone needs to use the lamp but finds it not operational, either because it is burned out or in the process of being replaced.

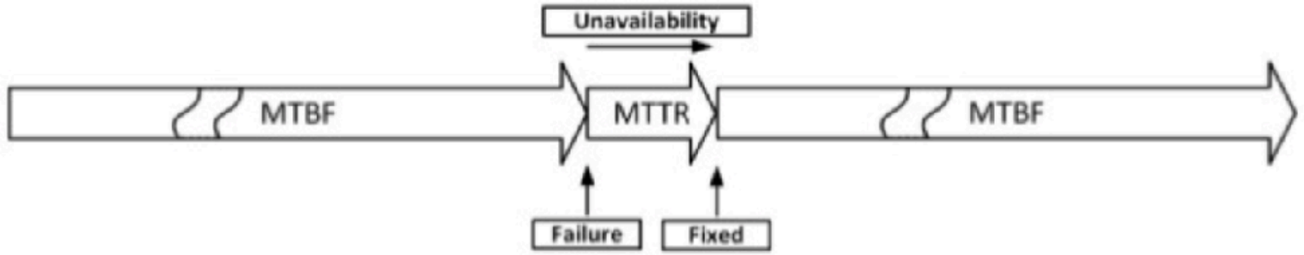
For a repairable system, the **operation time is not continuous** but consists of a series of up and down states.

Once the system fails, it is repaired and restored to its original operating condition.
The repetitive cycle of failure and repair is called an alternating renewal process.

The related random variables are the mean time between failures (MTBF) and the mean time to repair (MTTR).

$$Availability = \frac{MTBF}{MTBF + MTTR}$$

Mean time between failures over the sum of mean time between failures and mean time to repair.



Minimize MTTR
Automated redundancy and failover.
Spare parts on site.
Service contract with the supplier.

Reliability

From the Oxford Essential Dictionary of the U.S. Military, Oxford University, 2002.
Reliability - The **ability** of an item to perform a required **function** under stated **conditions** for a **specified time**.

From the McGraw-Hill Dictionary of Scientific and Technical Terms, 2003.
Reliability - The **probability** that a component, equipment, or system will satisfactorily perform its intended **function** under given **circumstances***, such as environmental conditions, operating time limitations, and maintenance frequency and thoroughness* for a specific **time**.

=> This is the most commonly used definition in engineering textbooks.

Key Terms
- **Ability/Probability.** Possession of the means or skills to do something.
- **Function.** An activity that represents/implements the purpose of a software project.
- **Circumstance/Condition.** A fact connected with or relevant to an event.
- **Time.** The progress of events allows for positioning them in order.

Reliability Engineering

The reliability function $R(t)$ shows the probability that a component or system will function without failure for at least time t.

$F(t)$ is the probability density function (PDF), or the likelihood that the failure will fall in a specified interval, such as a specific year.
$R(t)$ represents the survival function or the probability that something will survive past a specific time t.

The hazard rate (or failure rate) is the instantaneous probability of the first and only failure. It is a function $h(t) = \frac{F(t)}{R(t)}$.

The mean time between failures ($MTBF$) can be computed as $MTBF = \frac{total_uptime}{\#total_failures}$

It can also be seen as the expected value of the reliability function $R(t)$ as $MTBF = \int_0^{+\infty} R(t)\ dt$

The term $MTBF$ applies to **repairable systems**, while mean time to failure ($MTTF$) applies to **non-repairable systems**.

NB: While these metrics can quantify reliability in a certain moment, foreseen reliability properties in advance can be crucial to make informed decisions.

Reliability Engineering - Degradation Faults

The **Bathtub curve** describes a specific type of hazard function that consists of three parts:

- The first part shows a decreasing failure rate, called **early failures**.
- The second part features a constant failure rate, known as **random failures**.
- The third part exhibits an increasing failure rate, referred to as **wear-out failures**.

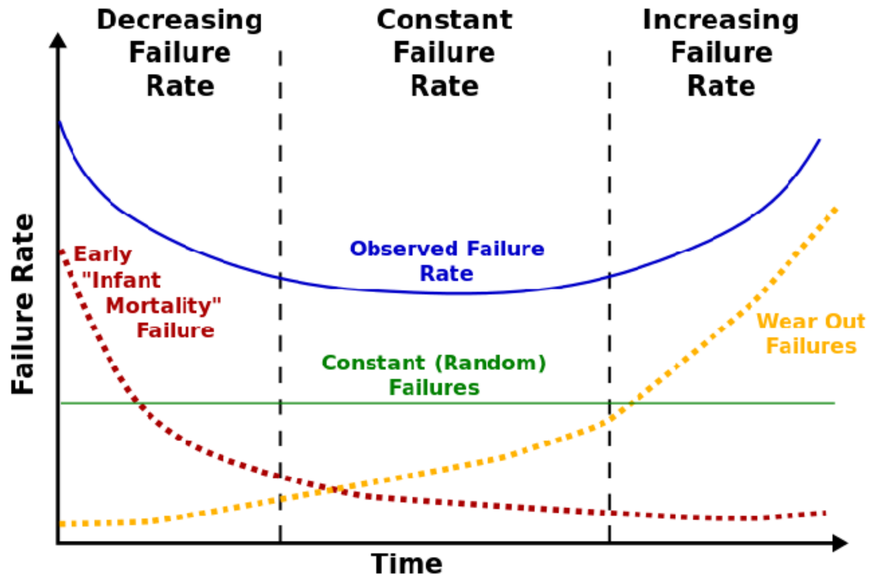
The concept is derived from human life, including infant mortality, failure probabilities, and a wear-out phase.
The curve assists in modeling failure rates because (i) they change based on the operation stage and (ii) the remediation strategies must be considered to effectively address the root causes.

Non-repairable items. This curve is mainly used to model the survivability of non-repairable items.

Repairable items. The failure rate of repairable items can change with time, but progressive repairs can improve reliability. An increasing failure rate occurs in repairable systems when wear-out failure of parts begins to predominate.

From a customer perspective, **early failures are unacceptable** because they are often caused by defects that are inherent to or built into a product. The manufacturer must find a way to **remove these defects before production**.

Appropriate specifications, **design tolerance**, and **component derating** should be employed. However, even the best design intentions can fail to consider all possible component interactions during operation.



Reliability Engineering - Accelerating Life Testing (ALT)

Stress testing should begin early in the development process to help identify design weaknesses and detect specific issues with assembly and materials.

Tests like these are called **HALT** (Highly Accelerated Life Test) or **HASS** (Highly Accelerated Stress Screen) and should be performed with increasing stress levels as necessary until failures occur.

Qualitative ALT: The primary goal of this ALT testing type is to gather information about failure modes, failure effects, and environmental stress limits, enabling the designer to understand how to rectify a fault.

=> The sample size is usually small and involves stressing a device to its extremes (e.g., temperature cycling from cold to hot to observe thermal fatigue).

Quantitative ALT: The primary objective is to collect live data by overloading the device's usage.

=> The sample size cannot be small and depends on the sampling strategy. Typical stresses include temperature, humidity, voltage, current, and pressure.

Software Reliability & Software Safety

Software **reliability** measures how **robust** a system is against defects, while software **safety** assesses the likelihood of a system causing **harm** to people or the environment.

Again, not all faults are the same!

We often describe IoT systems as cyber-physical by nature, meaning they include both hardware and software components.
Defects can arise at either level or during the integration process.

Software Security

What happens if the string received as input exceeds the right boundary of the buffer?
The data then gets written to a memory area that does not belong to the program variable referencing the buffer.

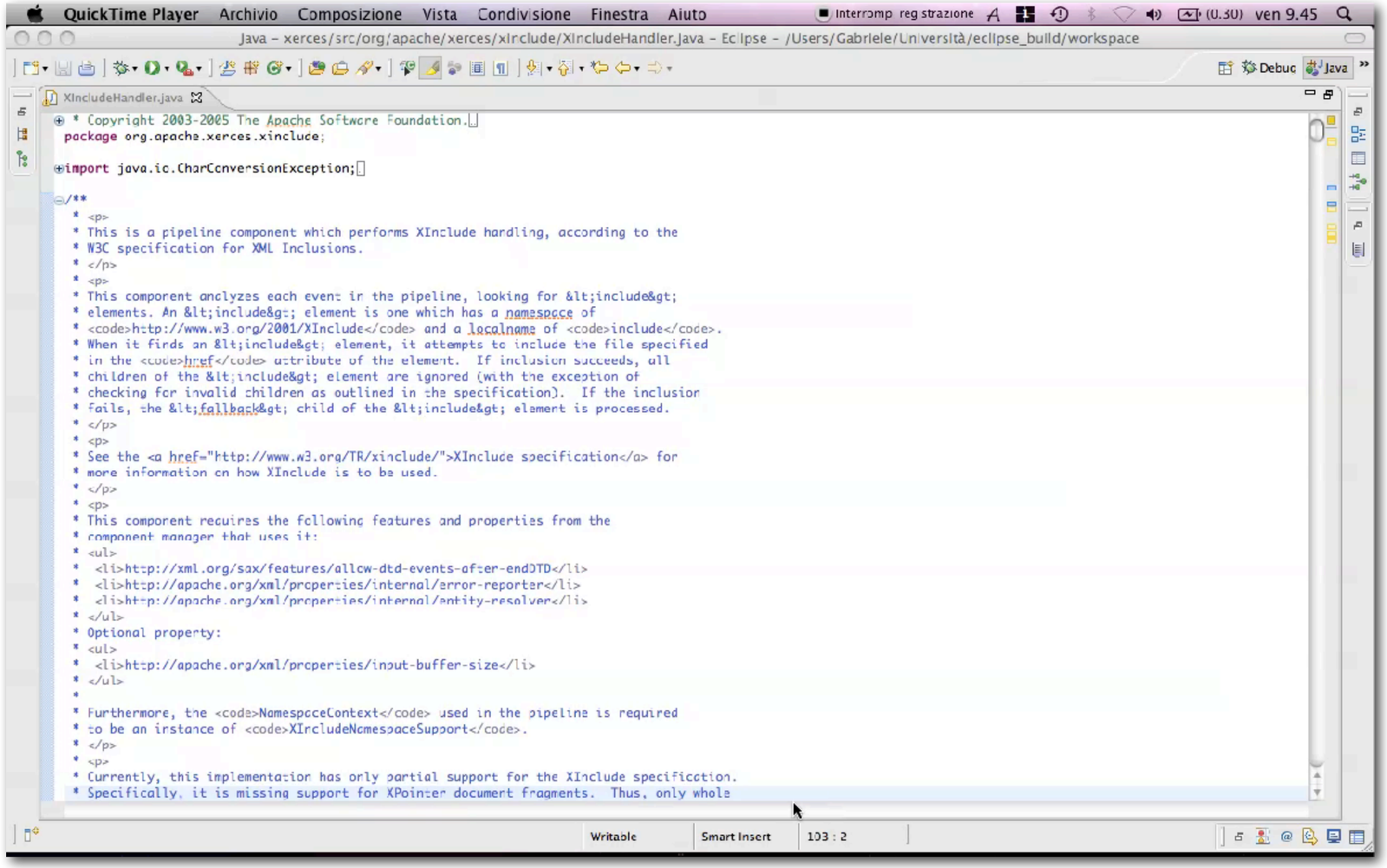
```
/* Do something…*/
void function(){

   char dst[256];
   char* s = read_string();
   strcpy(dst, s);

}
```

We will study software security by analyzing **vulnerabilities** using static and dynamic methods.

What else? Software Maintainability



Software Maintainability

During software maintenance and evolution, developers make **continuous changes** to add new features, adapt the system to new environments, and fix bugs (Lehman's laws!). Due to tight deadlines and stress, they often lack the time or willingness to design code modifications carefully.

This behavior can lead to **technical debt**, which is the future cost of rework required when opting for an easy but limited solution instead of a more comprehensive approach that may take longer.

Depending on its development stage, there are various types of technical debt, such as design debt, requirement debt, and test debt.

This course covers **testability**, which refers to the ease with which a system can be tested.

Related concepts are discussed in the Software Engineering Advanced Techniques and Sustainable Software Engineering courses.

Software Resilience

Software resilience measures how well a system can maintain the **continuity of its critical services** in the presence of **disruptive events** such as equipment failure and cyberattacks.

We will focus on performance testing, particularly on continuous integration and deployment.

What else?

- *Repairability* reflects the extent to which the system can be **repaired in case of failures**.
  Usually, maintainable software is easier to repair.
- *Error tolerance* reflects how user **input errors can be avoided and tolerated**.
- *Survivability* reflects the extent to which the system can deliver services under **hostile attack**.

Trust and Trustworthiness

The **dependence** of system A on system B represents the extent to which system A's dependability is (or would be) affected by that of system B. **Trust** is the accepted dependence.

Interaction among the dependability attributes

Maintainability	Reliability	Availability	Security
Reliability	Software dependability traits cannot be evaluated in isolation because they are closely connected. For instance, safe system operations depend on the system being available and functioning reliably.		
Availability	Similarly, a software system might be unreliable because its data has been corrupted by an external attack or due to poor implementation choices made by developers.		
Security	We frequently see techniques that tackle several dependability attributes at the same time.		




Break